MLFlow

For ML model lifecycle management

Plan

I. Introduction
II. Installation and configuration
III. Experiments tracking (MLFlow Tracking)
IV. Code packaging (MLFlow Projects)
V. Model deployment (MLFlow Models)
VI. Production model management (MLFlow Model Registry)
VII. Best practices



I. Introduction

Introduction

What is MLFlow

MLFlow is an open source platform for managing the lifecycle of Machine Learning models.

It allows users to **manage** the entire model development process, from data preparation **to production**.



Introduction

Why use MLFlow

MLFlow provides essential functionality for managing Machine Learning models:

- **Tracking** experiments: MLFlow allows you to track **settings**,**metric** and **artifacts** machine Learning experiments. This makes it easier to understand model performance and compare them easily.
- Management **models** MLFlow allows you to **register** and manage **versions** machine Learning models, making them easy to deploy in production.
- Integration into Machine Learning workflows: MLFlow can be integrated into a Machine Learning workflow, making it easier to optimize and regulate **hyperparameters** models.

Introduction

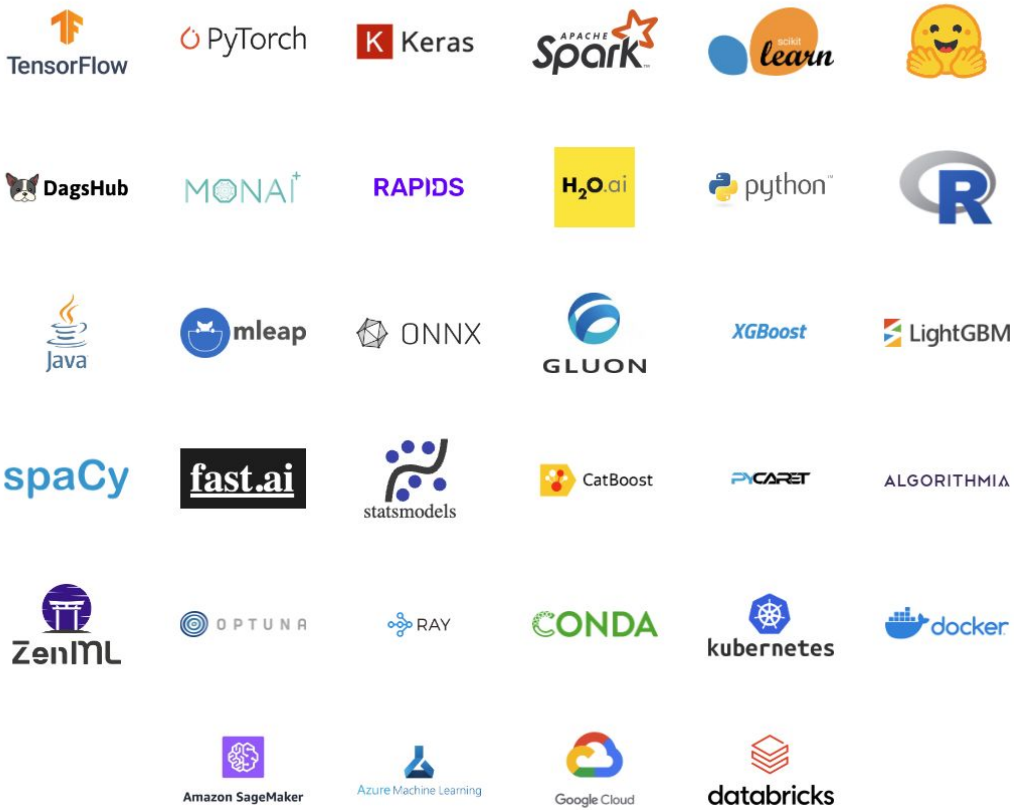
The functionalities of MLFlow

MLFlow's functionalities are divided into four main modules:

- **Tracking** : allows you to **follow** ML and **record parameters, metrics and artifacts** experiences
- **Projects** to structure ML's code into projects and make it easier to use **share** and their **execution**
- **Models** : to manage **packager** the ML models and **deploy in production**
- **Model Registry**: allows you to track and **manage model versions / tags** in a production environment



Integrations with:

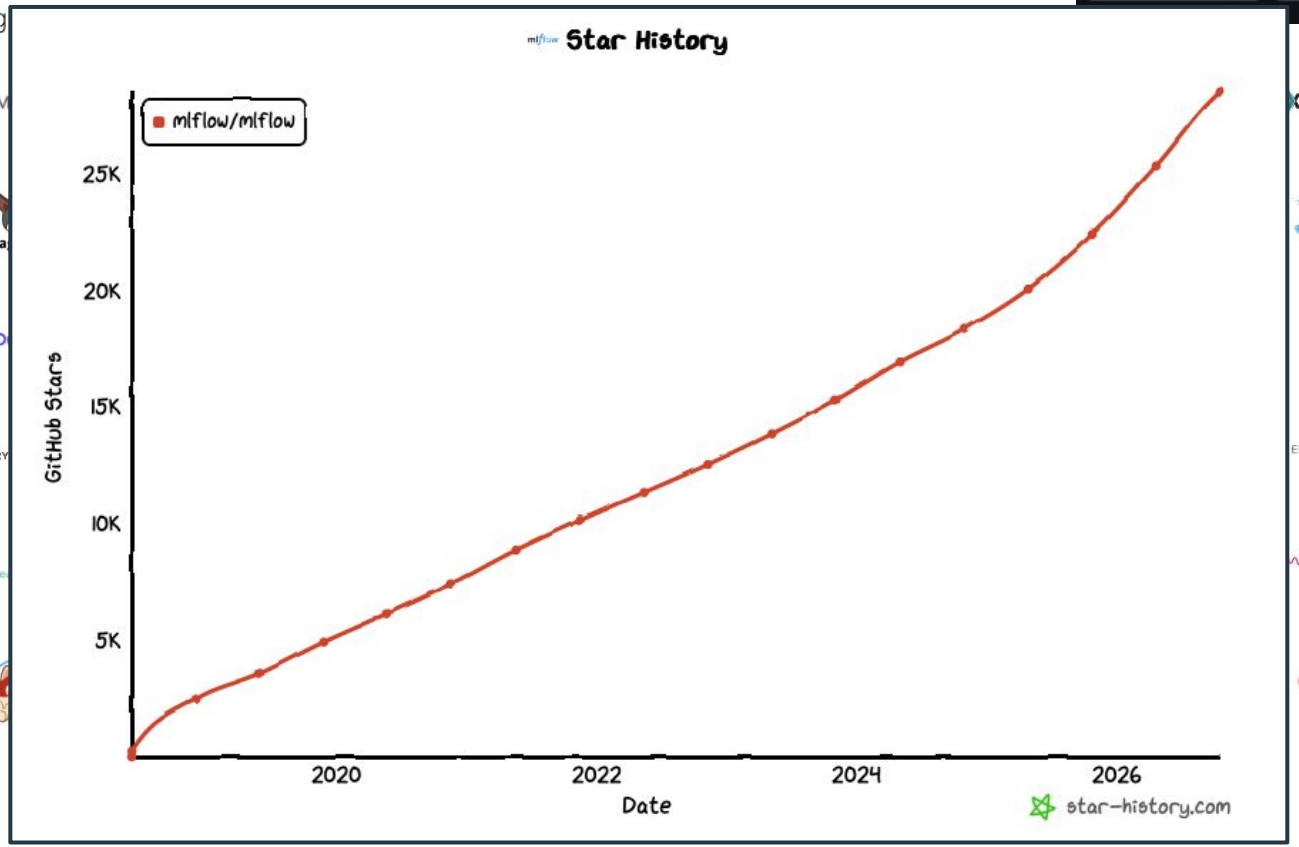


Fork 6.5k Star 28.5k

Organizations using and contributing to MLflow:



Organizations using



II. Installation and configuration

Installation and configuration

Installing the Python library

To use MLFlow, you first need to install it on your system and configure your development environment for its use.

MLFlow can be installed via pip, Python's package manager.

To install MLFlow, simply run the following command in your terminal:

```
# Install MLflow
pip install mlflow

# Install MLflow with extra ML libraries and 3rd-party tools
pip install mlflow[extras]

# Install a lightweight version of MLflow
pip install mlflow-skinny
```

Installation and configuration

Environment configuration

Once MLFlow has been installed, you can configure your development environment for its use. You can configure the following environment variables:

- MLFLOW_TRACKING_URI the URL of the MLFlow experience tracking server
- MLFLOW_EXPERIMENT_NAME the name of the current experiment

Installation and configuration

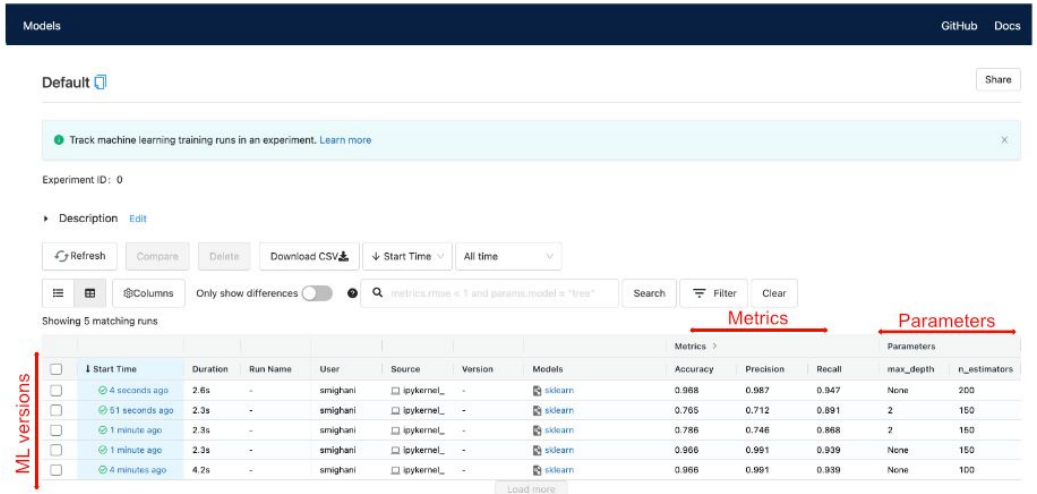
User interface

MLFlow features a user interface for viewing the results of recorded experiments

You can access it by executing the following command in your terminal:

```
mlflow ui
```

This will launch a local web server at the address http://localhost:5000 here you can view experiments recorded with MLFlow



Installation and configuration

In practice

```
# Installation de MLFlow
!pip install mlflow

import mlflow

# définition de l'URI de tracking
mlflow.set_tracking_uri("./results")

# définition du nom de l'expérience
mlflow.set_experiment("nom_experiment")
```

In this example, we have :

- Install MLFlow with the command **pip**
- Configured logging information by defining the tracking URI and experiment name using the **mlflow.set_tracking_uri()** and **mlflow.set_experiment()**

Note that the tracking URI is the place where information on MLFlow experiments will be **stored,** and can be a remote server or a local location. You can adjust the tracking URI to suit your needs

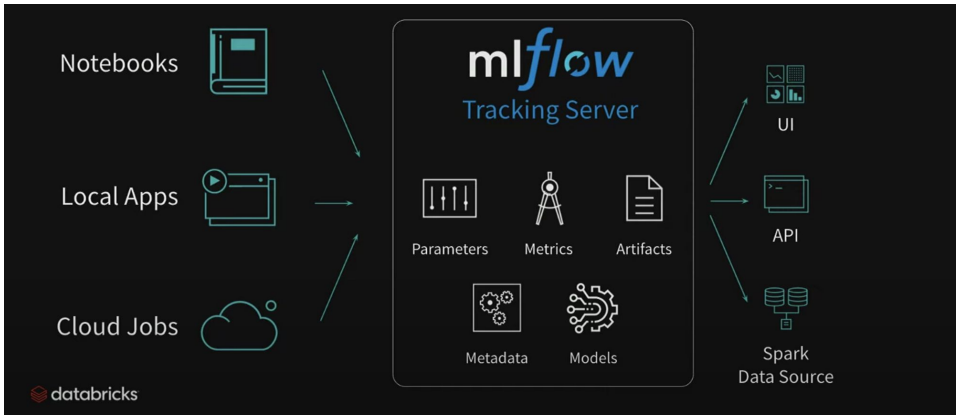
III. MLFlow Tracking : Tracking experiments



MLFlow Tracking

Experience tracking

The component **MLflow Tracking** is an API and user interface for recording and trackings **settings**, **metric** and **artifacts** machine Learning experiments. MLflow Tracking lets you record and query experiments using the Python API, REST API, R API and Java API



MLFlow Tracking

Experience tracking

It allows you to record and track :

- **settings** variables passed to your code at runtime. They can be hyperparameters, configurations or input variables
- **metric** performance indicators: performance indicators that measure the effectiveness of your model. They can include training and test metrics, such as accuracy, precision or recall
- **artifacts** files generated during the execution of your code, such as templates, graphics or data files.

MLFlow Tracking

Tracking metrics

To record the metrics of an ML experiment with MLFlow, you need to :

- Initialize an object mlflow.Run for the current experiment.
- Record metrics using the **mlflow.log_metric()**

```python
import mlflow
from sklearn.metrics import accuracy_score

# Enregistrer les métriques dans MLFlow
with mlflow.start_run():
    mlflow.log_metric(
        "accuracy",
        accuracy_score(y_test, y_pred)
    )
```

MLFlow Tracking

Parameter tracking

To save the parameters of a ML run with MLFlow, you must :

- Initialize an object mlflow.Run for the current experiment.
- Save parameters using **mlflow.log_param()**

```python
import mlflow

# Définir les paramètres
alpha = 0.5
l1_ratio = 0.2

# Enregistrer les paramètres dans MLFlow
with mlflow.start_run():
    mlflow.log_param("alpha", alpha)
    mlflow.log_param("l1_ratio", l1_ratio)
```

MLFlow Tracking

Artifact tracking

Artifacts are the files produced during the execution of a Machine Learning experiment, such as the **trained models** or the **datasets**

To record artifacts from an ML experiment with MLFlow, you need to :

- Initialize an object`mlflow.Run` for the current experiment.
- Record artifacts using the **mlflow.log_artifact()**

```python
import mlflow

# Enregistrer un fichier d'artefact dans MLFlow
with mlflow.start_run():
    with open("output.txt", "w") as f:
        f.write("Hello, World!")
    mlflow.log_artifact("output.txt")
```

MLFlow Tracking

… or use autologging

Automatic logging lets you record metrics, parameters and models without the need for explicit logging instructions.

The following libraries support automatic logging, with **mlflow.autolog()** :

- Scikit-learn
- Keras
- Gluon
- XGBoost
- LightGBM
- Statsmodels
- Spark
- Fastai
- Pytorch

```
import mlflow

from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor

mlflow.autolog()

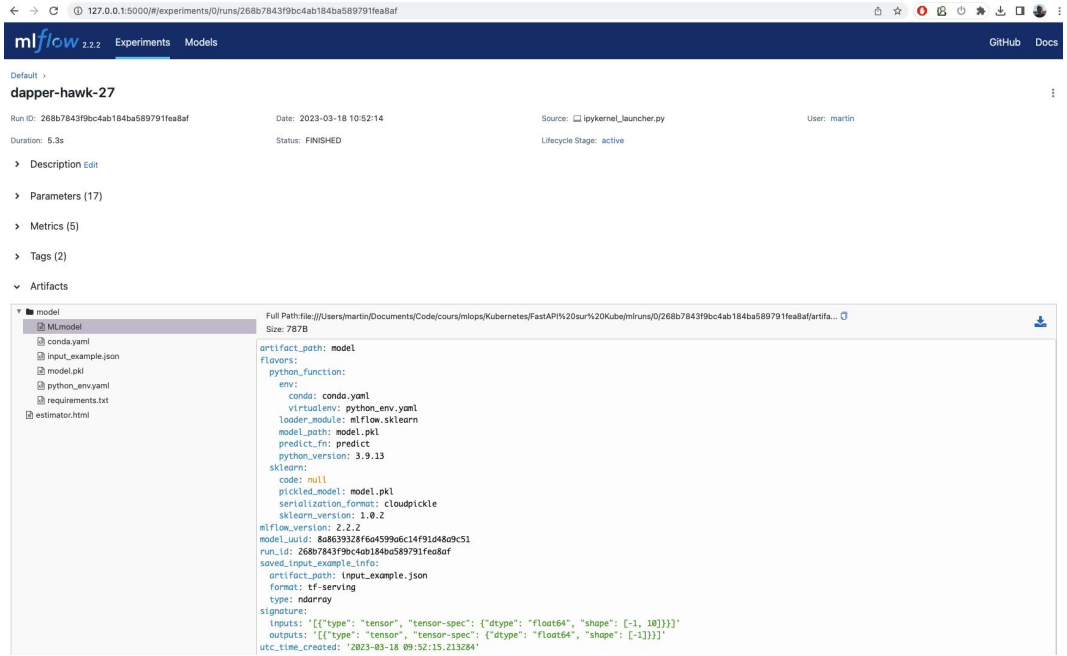
db = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(db.data, db.target)

# Create and train models.
rf = RandomForestRegressor(n_estimators=100, max_depth=6, max_features=3)
rf.fit(X_train, y_train)

# Use the model to make predictions on the test dataset.
predictions = rf.predict(X_test)
autolog_run = mlflow.last_active_run()
```

MLFlow Tracking

… or use autologging



IV. MLFlow Projects : Code packaging



MLFlow Projects

Code packaging

The module **MLFlow Project** structure ML's code into projects, making it easier to **share** and their **execution**

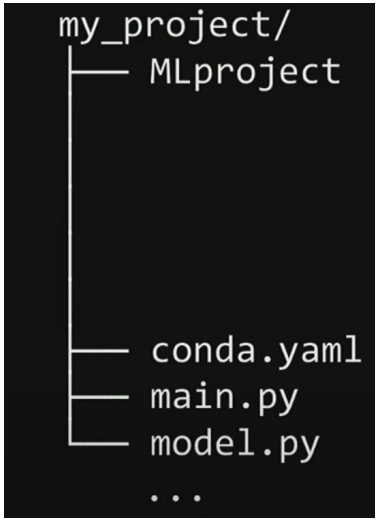
The benefits:

- easy reuse and reproducibility of the project
- API and CLI for running MLFlow projects
- links projects into workflows



MLFlow Projects

Code packaging



A project is just a directory or a Git repository containing your code
All you need to do is add an "MLproject" file: a simple yaml file at the root of your folder containing information on the environment and prerequisites.

```
conda_env: conda.yaml

entry_points:
  main:
    parameters:
      training_data: path
      lambda: {type: float, default: 0.1}
    command: python main.py {training_data} {lambda}
```

Find out more here:https://mlflow.org/docs/latest/projects.html

MLFlow Projects

Run code

It is possible to perform an MLFLow project and drive a template from a git repo or local project via the 'mlflow run' command:

```
mlflow run https://github.com/mlflow/mlflow-example.git -P alpha=5.0
```

If tracking is enabled, the results (parameters, metrics and artifacts) will be stored on the tracking API

MLFlow Projects

The MLFlow console

The MLFlow console is a command line interface for **interact with experience** recorded with MLFlow

You can access it by executing the following command in your terminal:

mlflow runs list --experiment-id=732924516436076370

This will display a list of experiments recorded with MLFlow, together with information on associated metrics, parameters and artifacts.

V.MLFlow Models : Deployment models



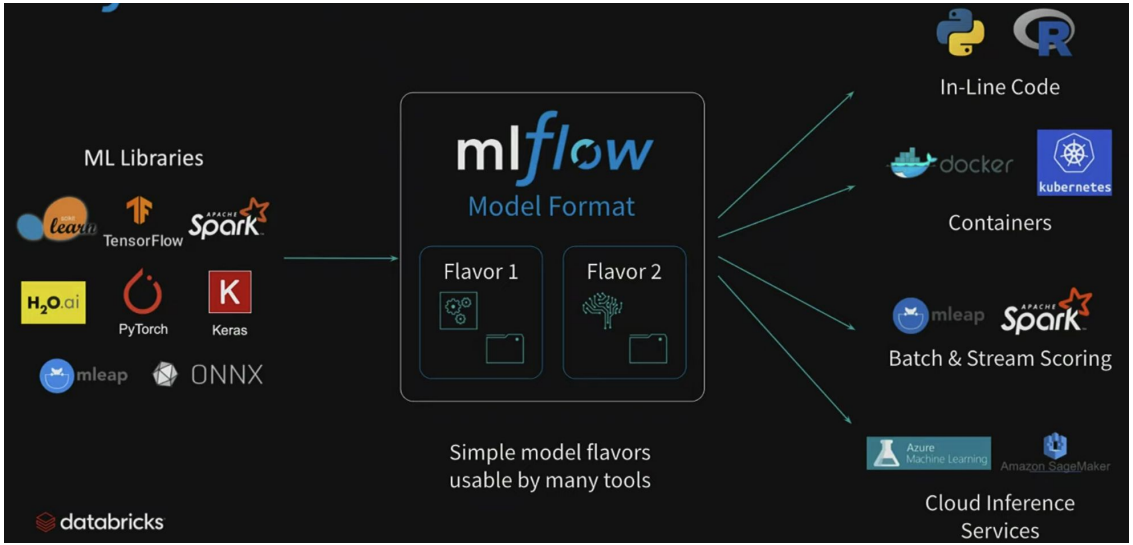
MLFlow Models

Model standardization

MLFlow offers model packaging and deployment functionstrained.

MLFlow Models will **standardize** the model driven with "**Flavors**"that will depend on the libraries used

Then there are standard functions to **operate** /**deploy** these standardized models



MLFlow Models

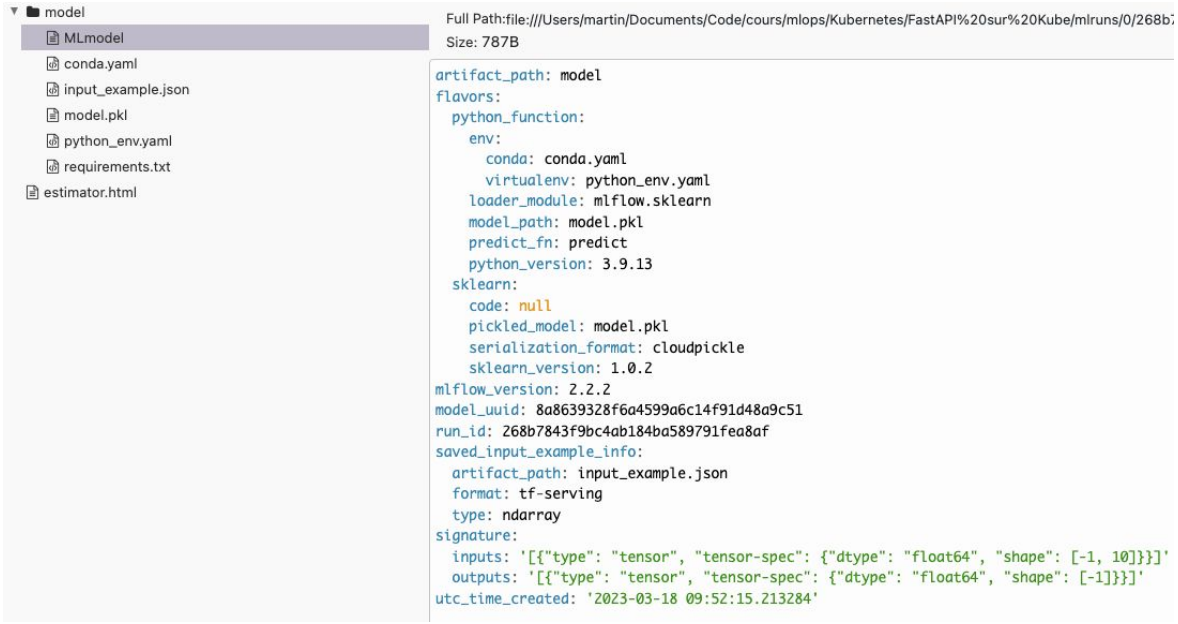
Saving the model

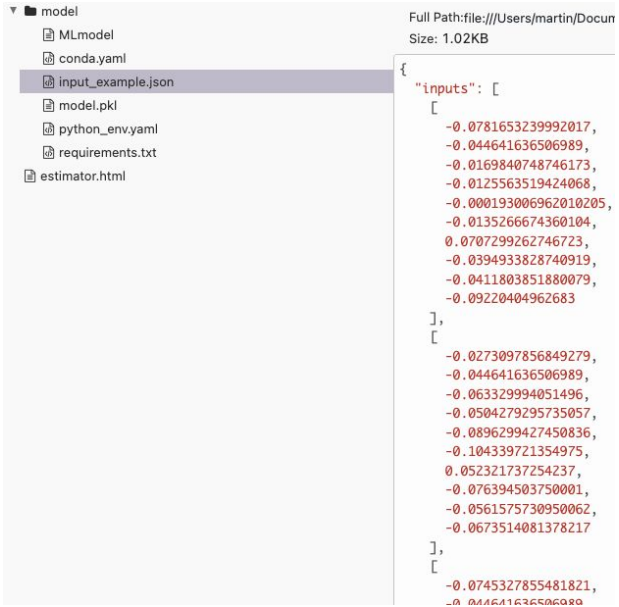
To save a model with MLFlow, you must :

- Register **model** using the mlflow.sklearn.log_model() method (if you're using scikit-learn) or the mlflow.tensorflow.log_model() method (if you're using TensorFlow)
- Specify **metric** and model parameters using the mlflow.log_metric() and mlflow.log_param() methods
- Register **artifacts** associated with the model using the mlflow.log_artifact() method

MLFlow Models

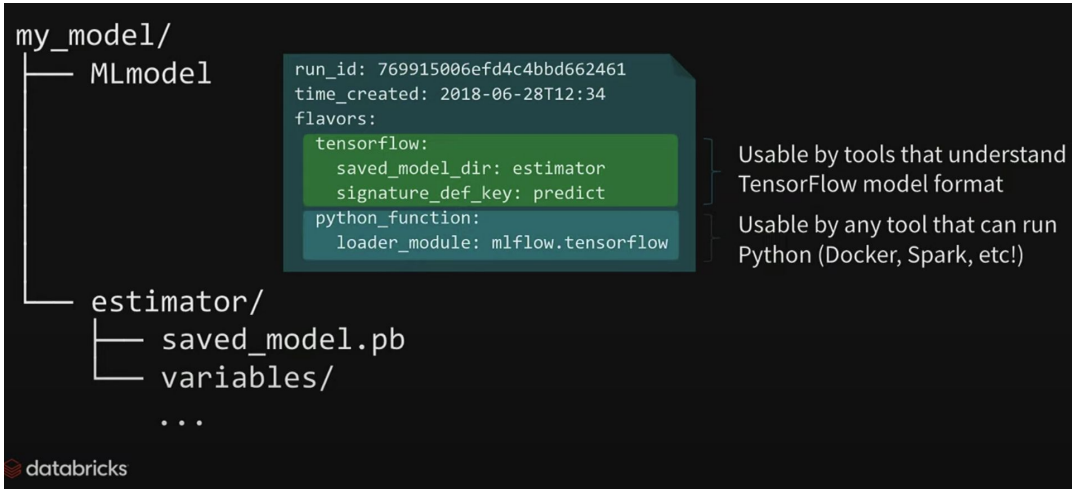
Saving the model

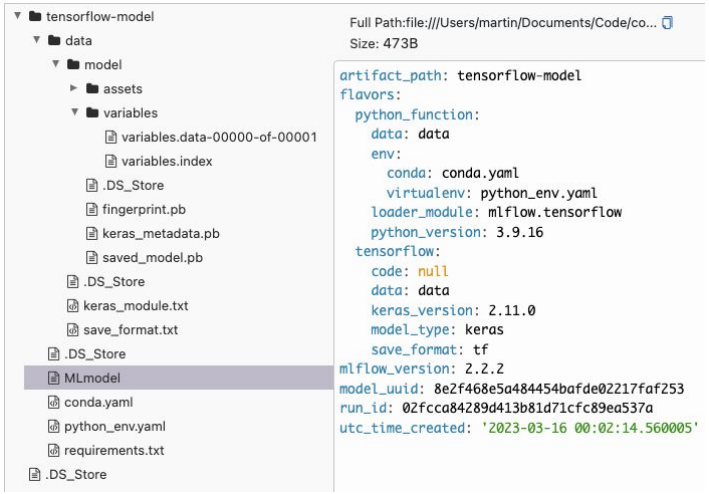




MLFlow Models

Saving the model





VI. MLFlow Model Registry : Model management in production



MLFlow Models Registry

Model management in production

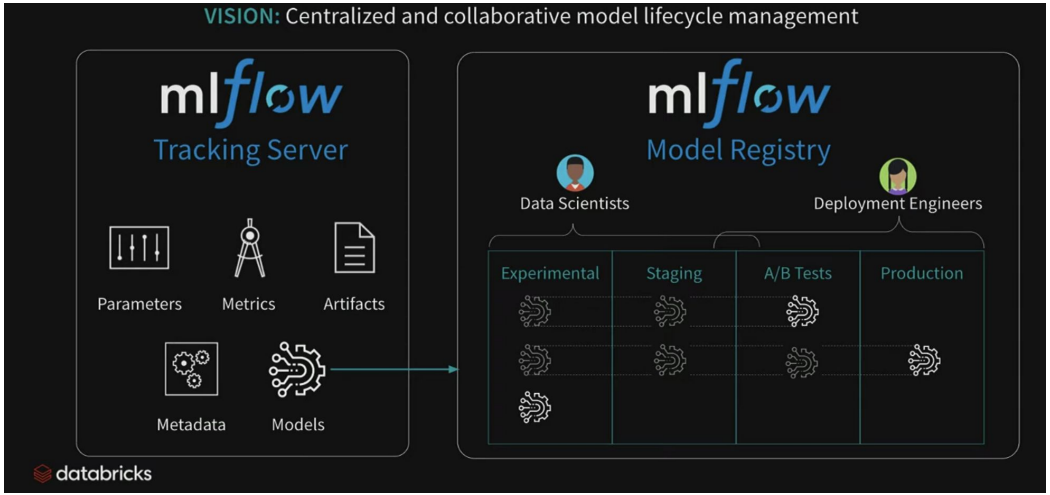
Module to **follow, manage** and **share** models :

1. **Recording models:** templates can be registered with their metadata (name, version, author, etc.) in the MLflow template registry
2. **Versioning models and history** each model registered in the register has its own **version number** unique, making it possible to **track changes** to the model over time. MLFlow conserves a history of registered models and their versions, enabling you to trace the evolution of models over time and return to an earlier version if necessary.
3. **Model management** MLflow's model registry allows you to manage registered models, organizing them into collections and **labeling** as ready for production, etc.
4. **Collaboration** save models: saved models can be shared with other members of the development team, facilitating collaboration on models
5. **Deployment** mLflow's model registry can be integrated with machine learning model deployment platforms, making it easy to deploy models in production
6. **Monitoring** mLflow's model registry enables you to track the performance of models in production, notably by monitoring metrics and prediction errors

MLFlow Models Registry

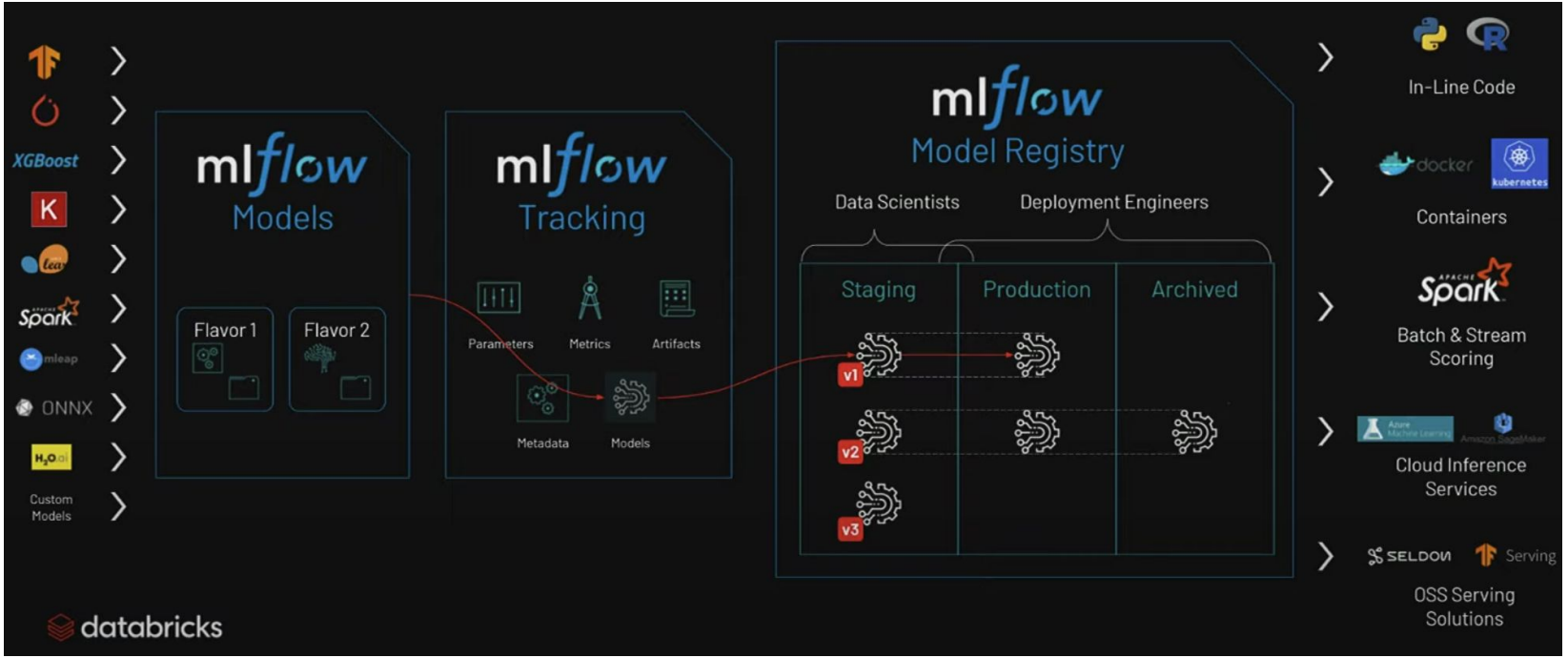
The link between trades

MLFlow pers to bridge the gap between the people who develop the model and those who will use it / put it into production:



MLFlow Models Registry

End-to-end vision, from tracking to registry



VII. Best practices

Best practices



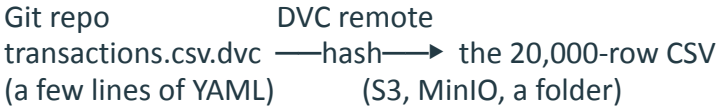
MLFlow can help you manage your Machine Learning projects more efficiently, but it's important to adopt the right practices to get the best results

- **organize** your **experiences** machine Learning in a clear and structured way. You can use **tags** and **annotations** to add additional information to your experiments, such as model name, training date or environment version
- follow **metric** which are **relevant** for your Machine Learning project. You need to select the metrics according to your project objectives and application domain
- **save hyper-parameters** used to train your model. This will enable you to reproduce results more easily and compare the performance of different models
- **document** the models you've trained, providing information on their architecture, performance and limitations

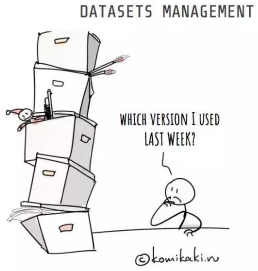
VIII. Data Version Control

What is DVC ?

- Git is built for text files. It chokes on large data files (CSV, images, model weights)
- DVC keeps the big files outside Git and tracks them through a small pointer file
- Git stores the pointer (an md5 hash) and DVC stores the bytes in a remote (a folder, S3, GCS, Azure, MinIO)
- Result: a Git commit hash identifies the code and the exact data.



Why do we need it ?



Questions you'll get months after training a model:

- Which data was this model trained on?
- Can you rebuild it exactly?
- What did the data look like before the last refresh?
- Without data versioning, data_final_v2_REAL.csv is the only answer.

DVC makes data:

- Reproducible: same commit gives same data and same model
- Shareable: a teammate runs dvc pull, not "I'll send you a zip"
- Auditable: the data's fingerprint is in the Git history
- Reversible: roll back code and data together

When is it useful ?



- Use it when:
  - Data is too big for git (about 50MB)
  - Datasets are refreshed or relabelled over time
  - Several people work on the same data
  - You must audi or reproduce a model (health -> need for transparency for example)
  - Data drift forces retraining

- Skip it when:
  - dataset is small and rarely changes
  - data comes from a database you can query by date

Set it up in practice




```
git init && dvc init
dvc remote add -d storage $HOME/dvc-storage
dvc add data/transactions.csv
dvc push
add -A && git commit -m "january data" && git tag v1-jan
```

- `dvc init` creates the .dvc/ folder (commit it)
- `dvc remote add -d`: sets the default storage
- `dvc add`: moves the file into the cache, writes the .dvc pointer and adds the data file to .gitignore
- `dvc push`: uploads the cache to the remote.

Lab session